

利用大语言模型实现高效且个性化的智能家居自动化

Chaerin Yu, Chihun Choi, Sunjae Lee, Hyosu Kim, Steven Y. Ko, Young-Bae Ko 以及 Sangeun Oh

摘要——智能家居设备的激增增加了控制和管理它们的复杂性，导致用户疲劳。在此背景下，大语言模型通过为物联网控制提供自然语言界面，提供了一个有前景的解决方案。然而，现有基于大语言模型的方法因大语言模型的非确定性、高推理延迟和成本以及有限的个性化，而存在设备控制不可靠和低效的问题。为了应对这些挑战，我们提出了IoTGPT，一个基于大语言模型的智能家居代理，旨在以可靠、高效且个性化的方式执行物联网指令。受人类管理复杂任务方式的启发，IoTGPT将用户指令分解为子任务并记忆它们。通过复用已学习的子任务，后续指令可以用更少的大语言模型调用次数得到更高效的处理，从而提高可靠性并降低延迟和成本。IoTGPT 还通过使单个子任务适应用户偏好来支持细粒度个性化。我们的评估表明，IoTGPT 在准确率、延迟/成本和个性化方面均优于基线，同时减少了用户工作量。

在 VER I. 引言

过去的十年中，物联网技术的发展已将各种设备——从照明、供暖到安全系统和家用电器——引入智能家居，使得这些环境日益复杂。截至2025年，美国家庭平均拥有21台联网的物联网设备 [1]，这突显了其复杂性的规模。随着互联设备数量的不断增长，用户面临着更高的认知负荷，需要有效地控制和管理它们 [2]。因此，市场对高效且直观的物联网设备控制方法的需求日益增长，以帮助减轻用户的疲劳感。

不出所料，人们已经探索了各种方法来促进智能家居自动化。现有的物联网平台，如三星SmartThings [3]、Google Home [4] 和Apple HomeKit [5]，提供了可视化界面来定义触发-动作规则，而无需编写源代码。虽然此类界面对于处理简单任务很直观，但随着所需自动化的复杂性增加，它们会施加越来越大的认知负担。或者，语音助手，如Google Assistant [6]

C. Yu 和 Y. Ko 隶属于韩国水原市16499号，亚洲大学人工智能融合网络系（电子邮件：dbcofls6961@ajou.ac.kr; youngko@ajou.ac.kr）。S. Lee 隶属于韩国水原市16419号，成均馆大学计算机科学与工程系（电子邮件：sunjae.lee@skku.edu）。H. Kim 隶属于韩国首尔市06974号，中央大学计算机科学与工程学院（电子邮件：hskimhello@cau.ac.kr）。S. Y. Ko 隶属于加拿大不列颠哥伦比亚省本拿比市V5A 1S6号，西蒙弗雷泽大学计算科学学院（电子邮件：steveyko@sfu.ca）。C. Choi 和 S. Oh 隶属于韩国首尔市02841号，高丽大学计算机科学与工程系（电子邮件：clgns0102@korea.ac.kr; sangeunoh@korea.ac.kr）。Sangeun Oh 和 Young-Bae Ko 是共同通讯作者。

和苹果Siri [7]，可以将用户的口头指令翻译成物联网指令。这些系统在解释基于预定义指令模板的明确且结构良好的请求（例如，“晚上6点打开客厅的灯”）时表现良好。然而，它们难以处理更自然但模糊的指令（例如，“日落后让客厅变亮”），这些情况需要推理用户意图，从而导致不正确或不完整的自动化。

大语言模型（LLMs）正在成为智能家居自动化领域的变革者。与传统方法不同，基于大语言模型的智能体 [8]–[12] 能够灵活地解释用户话语并推理潜在意图，甚至能从模糊的指令中推导出物联网指令。然而，它们也面临关键限制。首先，大语言模型的非确定性本质损害了可靠性：幻觉指令需要用户反复审查和修正结果。其次，大语言模型推理计算成本高昂，提示词处理和推理过程漫长。现有的智能体通过多步推理和重新规划反复调用大语言模型，加剧了这种成本；在我们的实验中，一种方法处理一条简单指令耗时长达151秒。最后，大语言模型缺乏用户特定上下文，难以实现个性化自动化。例如，对于像“让卧室准备好睡觉”这样未明确指定的指令，用户通常对其具体含义（例如，调暗照明、降低温度或减少背景噪音）有不同的偏好，而当前系统难以可靠地推断这些偏好。

为了克服这些限制，我们引入了IoTGPT，这是一个基于大语言模型的智能家居代理，它能以可靠、高效且个性化的方式将自然语言指令翻译成可执行的物联网指令。其核心理念是任务分解与记忆化。受人类将复杂任务分解为更小、更易管理的子任务并学习以备将来使用的启发，[13], [14]，IoTGPT 将一条指令分解为明确的子任务（例如，“让卧室准备好睡觉” → {“调节空调温度，” “调暗睡眠灯”}），这些子任务可以轻松地转化为API级别的指令。然后，这些子任务及其翻译后的设备指令会被存储在记忆中，并在未来指令中出现类似设备控制子任务时被复用。

这种分解带来了两大好处：i) 子任务可以被记忆并在不同指令间复用，从而减少重复的大语言模型推理，进而缓解指令生成中的不可靠性、延迟和成本；ii) 通过子任务暴露的可配置参数，允许基于给定的上下文和用户偏好对底层设备指令进行微调。此外，在一个子任务中学到的偏好可以指导其他子任务。例如，如果IoTGPT 了解到某用户对“凉爽”的偏好对应22°C（在控制

空调时)，那么它在使用其他制冷设备（如风扇）调节室温时，也能自动应用这一相同偏好，从而实现即使设备配置发生变化也能自适应个性化。

为实现我们的目标，IoTGPT 引入了三个关键特性：*i)* 可靠的子任务分解：由于其随机性，大语言模型可能生成不一致的子任务或无效的设备指令。为缓解此问题，IoTGPT 将结构化提示与校正机制相结合，以确保指令有效。*ii)* 高效的子任务复用：为促进高效的子任务复用，IoTGPT 将任务知识组织成一个抽象的、分层的数据结构（任务记忆）。任务记忆以通用形式紧凑地存储子任务和设备指令，从而实现跨任务的灵活复用。*iii)* 自适应个性化：为了即使在设备可用性不同的情况下也能实现个性化，IoTGPT 将用户偏好从设备特定的设置中抽象为与设备无关的环境属性。这使得偏好能够在设备间迁移；例如，如果从空调使用中学习到用户偏好凉爽环境，IoTGPT 在仅有风扇可用时，可以通过调整风扇风速来近似实现该偏好。

贡献。总而言之，我们的贡献如下：

- 我们提出了IoTGPT，这是一种基于大语言模型智能家居代理，它利用任务分解和记忆化来实现可靠、高效且个性化的物联网控制。
- 我们引入了一种分层式的任务记忆，它支持高效的子任务复用，减少对大语言模型的依赖，从而提升自动化任务的可靠性和效率。
- 我们设计了自适应个性化方案，将用户偏好抽象为与设备无关的环境属性，从而实现在异构设备配置间的迁移。
- 我们在三星SmartThings平台上对IoTGPT的原型进行了实验，结果表明，与最先进的基线相比，其任务成功率最高可提升85.43%，延迟降低78.40%，成本降低44%。
- 我们与15名参与者进行了深入的用户研究，以评估IoTGPT。结果显示，与现有方法相比，它降低了认知负荷，提升了可用性，并实现了更有效的个性化。

二、相关工作

IoTGPT 是一个用于智能家居自动化的基于大语言模型的智能体。基于此范围，我们从三个领域回顾相关工作：传统智能家居自动化、基于LLM的智能家居代理和基于LLM的任务自动化。

传统的智能家居自动化。自动化已成为智能家居中快速增长的需求，推动了各种自动化技术 [15]的发展。传统平台（例如，SmartThings [3]、Google Home [4]、Apple HomeKit [5]）要求用户通过可视化界面手动创建触发-动作规则。这些规则虽然简单，但随着任务复杂度的增加，会限制灵活性并带来较高的认知负荷。为了减轻这种负担，基于众包的解决方案 [16]、[17]在用户间共享规则，但由于质量控制和标准化方面的挑战，存在可靠性问题。机器学习方法 [18]试图通过从传感器

和设备日志推断用户习惯来实现个性化控制，但往往无法捕捉用户意图。自然语言界面 [19]–[21] 也已集成到Siri [7] 和Google Assistant [6]，等助手中，通过语音指令提供直观控制。然而，它们对预定义指令模板的依赖，限制了其对非正式表达的适应能力，这些表达往往具有歧义，从而降低了可用性。因此，传统方法仍然僵化且繁琐，难以处理复杂或模糊的请求。这凸显了对能够自主处理意图解释和适应、最大限度减少对用户手动干预依赖的系统的需求。

基于大语言模型智能家居代理。近期研究已探索利用大语言模型进行智能家居自动化 [8]–[12]。一些工作 [10]、[11] 通过大语言模型的推理能力将多样的自然语言请求转化为自动化规则，而 [12]则利用大语言模型提供可视化选项以消除指令歧义。然而，这些方法需要用户大量参与，要求用户要么验证生成的规则，要么手动选择预期解释。Sasha [8] 通过多步推理（如用户请求澄清和相关设备筛选）减轻了这一负担。但它仍然缺乏对用户上下文和个性化的考量。此外，其整体式方法——在单次推理过程中处理整个指令——使其容易在处理复杂任务时出现故障。为解决此问题，SAGE [9]动态规划推理模块序列以灵活处理任务，并利用过去的设备交互实现个性化。然而，这种对大语言模型的严重依赖，因其非确定性本质，既带来了计算开销，也导致了错误累积。再者，其设备特定的个性化在设备配置变更时适应性受限。相比之下，IoTGPT通过将指令分解为可复用的子任务并缓存以供将来使用，最大限度地减少了重复的大语言模型推理。此外，与先前局限于设备特定历史记录的工作不同，IoTGPT将用户偏好抽象为与设备无关的属性，从而能够在异构设备间实现灵活的个性化。

基于LLM的任务自动化。大语言模型已被探索作为不同计算领域中端到端任务自动化的解决方案。例如，移动代理 [22]–[26] 通过利用大语言模型解释GUI组件来自动控制移动应用。另一系列工作 [27]–[30] 侧重于通过分析网络组件（如HTML DOM树结构）来实现网络浏览器中的任务自动化。类似地，大语言模型通过监控用户屏幕 [31]–[34]，实现了桌面生产力软件内的操作自动化。这些研究展示了大语言模型通过屏幕解释在软件环境中实现任务自动化的潜力。值得注意的是，与IoTGPT类似，其中一些工作采用了任务分解策略来处理复杂的工作流。然而，IoTGPT 所针对的智能家居自动化引入了独特的挑战：它需要对动态物理上下文（例如环境条件）进行推理、协调异构设备，并支持自适应个性化。IoTGPT通过将任务分解与上下文感知记忆相结合来应对这些独特的复杂性，从而确保物理空间中可靠且个性化的自动化。

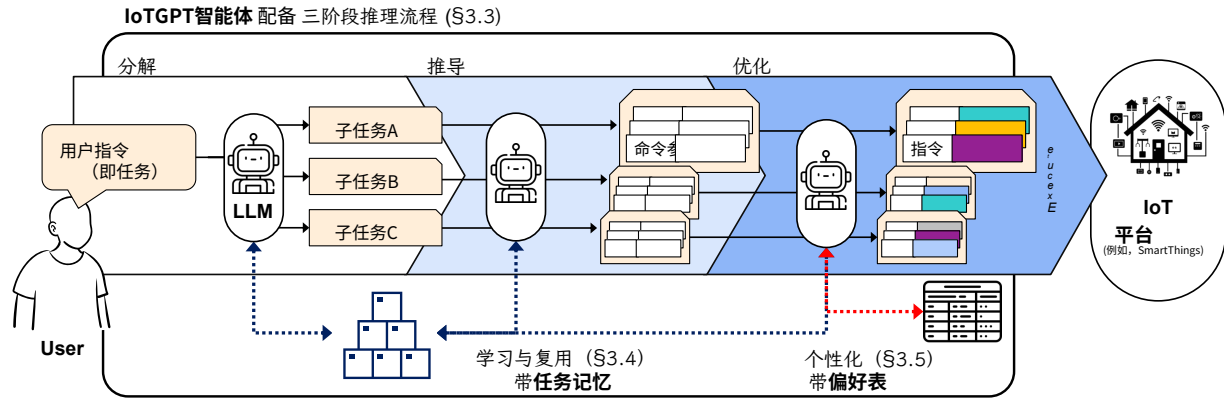


图 1. IoTGPT 系统概览，该系统通过一个三阶段推理流程（分解-推导-优化）将自然语言指令翻译为物联网指令，并利用任务记忆进行学习/复用，以及利用偏好表实现个性化。

III. IOTGPT 设计

我们提出IoTGPT，一个基于大语言模型智能家居代理，旨在实现可靠、高效和个性化的物联网控制。本节详述其设计原则与挑战。

A. 系统设计概述

设计原则。面对复杂且模糊的任务时，人们很少能一步到位地解决；相反，他们会将其分解为更小、更易管理的子任务 [13][14]。受此认知过程的启发，我们设计的IoTGPT 以类似方式处理用户指令，实现智能且个性化的智能家居自动化。图1展示了IoTGPT的整体设计。其核心是一个三阶段推理流程，逐步将自然语言指令转化为可执行的物联网指令，例如触发-动作规则、直接控制命令或设备查询¹。每个阶段都利用大语言模型来逐步优化中间结果，最终生成能有效反映用户偏好的个性化输出。这三个阶段如下：

●**分解：**给定一条用户指令，IoTGPT 会将其分解为若干个子任务，这些是具体的、设备相关的操作，共同完成该指令。如果指令本身模糊（例如“让卧室准备好睡觉”），IoTGPT会利用大语言模型的常识来提出合理的子任务作为基线（例如，{“调节空调温度”、“设置加湿器级别”、“调暗睡眠灯”}）。此提议仅作为起点而非最终决定，因为初始猜测可能无法完全匹配用户的真实意图。

●**推导：**对于每个子任务，IoTGPT 会生成带有抽象占位符（用于未指定参数）的可执行API级设备命令（例如，“调节空调温度” $\Rightarrow$ [打开空调 $\rightarrow$ 设置模式为 [模式值] $\rightarrow$ 设置温度为 [温度值]]）。此阶段侧重于生成有效的命令结构，同时保持未指定参数可配置。

¹触发-动作规则是事件驱动的自动化（例如，“当有人在客厅时打开灯”）。直接控制命令是即时的设备操作（例如，“让卧室准备好睡觉”）。设备查询请求智能家居状态（例如，“房间温度是多少？”）。

●**优化：**IoTGPT 通过根据从上下文和过往交互中推断出的用户偏好来填充参数，从而个性化这些命令。然而，如果偏好证据不可用（例如，首次使用），IoTGPT会采用保守默认值，并将不确定的选择标记出来供用户审核。

关键的是，在精炼阶段之后，IoTGPT 会进入一个人机协同步骤，用户可以在此审查提案，并在需要时修改子任务（例如，添加“播放音乐”）或调整参数以匹配其偏好。此设计通过将干预整合到一次最终审查中，而非要求逐步监督，从而最大限度地减少了用户负担。此外，此反馈会更新系统学习到的任务知识（即，任务记忆）和偏好配置文件，以便后续指令能更好地反映用户偏好。一旦最终确定，生成的命令将被发送到物联网平台（例如，三星 SmartThings）。触发-动作规则在其条件满足时被安装并执行，而直接控制命令和设备查询则立即执行。

任务分解的优势。遵循此架构，我们的任务分解方法提供了两大主要优势。首先，它实现了跨任务知识的有效复用，从而提升了效率和可靠性。为实现这一优势，IoTGPT利用了其任务记忆，该记忆记录了先前学习过的子任务及其相关的命令结构。例如，一旦IoTGPT 在处理一条指令时推导出了“调节空调温度”这一子任务的设备级命令，它便可以在处理另一条指令（例如“保持厨房凉爽”）时复用此知识，同时根据上下文（例如烹饪）调整参数，而无需从头重新计算。随着任务记忆的增长，IoTGPT 对重复的大语言模型推理的依赖减少，从而缓解了物联网指令生成中的不可靠性和延迟问题。

其次，任务分解使得在子任务级别实现细粒度的个性化成为可能。由于子任务通过其低级命令暴露了可配置的参数，IoTGPT可以根据用户偏好来调整这些值。此外，一旦学习到的用户偏好可以灵活地迁移到未来任务的类似子任务中。例如，如果用户的“凉爽”环境偏好已通过“调节空调温度”这一子任务习得，那么当空调不可用时，同样的偏好之后也可以应用于另一个同样

影响温度的（子任务），例如“调节风扇风速”。这种子任务级别的个性化使得IoTGPT能够捕捉随时间累积的偏好，并将其应用于新任务。

B. 技术挑战

尽管任务分解带来了明显的好处，但将这一设计付诸实践仍面临若干技术挑战：

C1. 如何可靠地分解任务？生成有效物联网指令的一个关键挑战在于，大语言模型常常无法稳定地将自然语言指令分解为子任务并推导出其底层设备指令。这是因为大语言模型容易产生歧义和非确定性：当被要求“让卧室准备好睡觉”时，它们可能会引入在当前环境中无法执行的子任务（例如，在没有安装智能百叶窗的情况下“调节百叶窗”），或者生成无效的设备指令（例如，为空调设置不支持的<mode>模式</mode>）。此类行为可能导致最终生成的物联网指令不完整或不可行。因此，确保可靠的任务分解需要精心设计的提示策略和适当的校正机制（将在第四节讨论）。

C2. 如何高效支持子任务复用？即使子任务及其设备指令被正确生成，由于上下文的变化，跨任务复用它们仍然具有挑战性。相同的子任务（例如“调节空调温度”）可能出现在不同的上下文中，比如“保持厨房凉爽”与“让卧室准备好睡觉”。如果每个特定于上下文的变体都被单独存储，任务记忆会迅速膨胀，导致过高的空间开销，并使得在类似情境下复用知识变得困难。另一方面，如果子任务在记忆时脱离了上下文，复用可能会忽略上下文细微差别，无法恰当地反映个性化。因此，高效的复用需要一个设计良好的任务记忆，能够对先前存储的任务知识进行上下文感知适配。这涉及选择性检索和调整相关的任务知识，以适应当前情境（将在第五节讨论）。

C3. 如何自适应地个性化物联网指令？传统的智能家居系统个性化依赖于设备交互历史来捕获用户偏好，通常是在单个设备级别。然而，应用这些设备特定的偏好存在两个局限性。首先，为一个设备捕获的偏好无法轻易转移到具有不同功能的其他设备上；例如，知道用户偏好的空调设置并不能直接指示当风扇是唯一可用选项时，应如何配置风扇。其次，由于偏好依赖于上下文，仅依赖设备使用历史很难在不同的上下文中应用正确的偏好。为了克服这些限制，IoTGPT将用户偏好抽象为与设备无关的环境属性（例如，温度、亮度），并为不同的上下文（例如，睡眠与锻炼）分别维护它们。这种上下文感知的抽象实现了偏好不同异构设备和上下文之间的灵活转移（将在第六节讨论）。

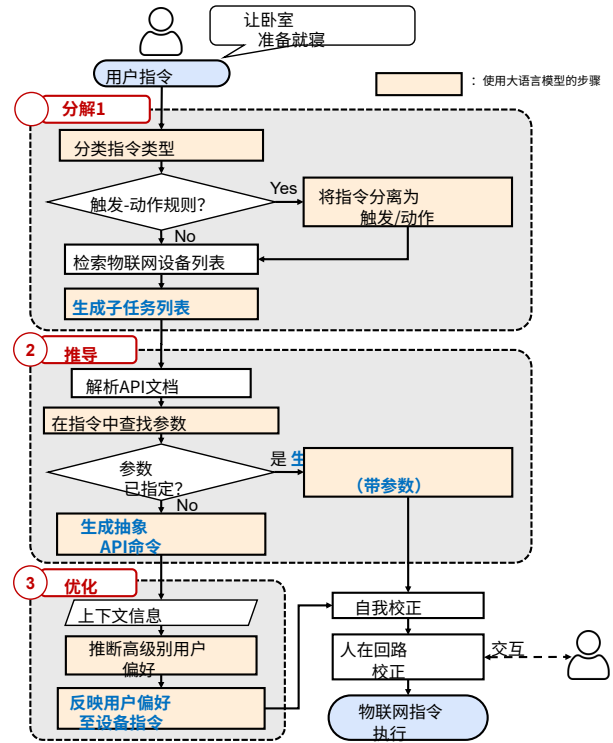


图2. IoTGPT流水线的工作流程

IV. 任务分解流水线

为了将自然语言指令翻译成物联网指令，IoTGPT 采用了分解-推导-优化流水线，如图2所示。本节概述了该流水线如何运作，以及IoTGPT 如何解决大语言模型固有不可靠性所引发的问题。

A. 流水线的三个阶段

分解阶段。分解阶段的目标是从给定的用户指令生成一个可执行的子任务列表。基于大规模通用知识训练的大语言模型，可以通过常识推理推断出具有可接受准确率合理子任务。为实现此目标，IoTGPT首先提示大语言模型将指令分类为三种类型之一：触发-动作规则、直接控制命令或设备查询。一旦类型被确定，IoTGPT 会通过物联网平台的API调用，检索当前连接到环境中的物联网设备列表。此列表包含设备及其可用功能和规格，并作为上下文提供给大语言模型。通过将推理过程建立在真实的设备列表之上，IoTGPT降低了引入无法在当前环境中执行的子任务的风险。在指令类型和设备信息的引导下，大语言模型随后生成完成指令所需的子任务，以及执行这些子任务所需的设备。特别是对于触发-动作规则，大语言模型被提示首先将指令拆分为触发部分和动作部分，然后为每个部分生成相应的子任务。

例如，给定指令“让卧室准备好睡觉”，该指令在指定应采取哪些具体行动方面有些模糊，但大语言模型仍可利用

常识知识来推断出一组合理的子任务。分解阶段的输出将是一个JSON格式的子任务列表，例如“调节空调温度”、“设置加湿器级别”和“调暗睡眠灯”，每个子任务都与相关设备关联，如下所示：

```
{ "CommandType": "直接控制命令", "Action": { "name": "
```

推导阶段。推导阶段的目标是将从前一阶段获得的每个子任务，翻译成严格符合物联网平台API规范的一系列底层设备指令。为此，IoTGPT会向大语言模型提供三个输入：*i)* 用户指令，*ii)* 子任务列表，以及*iii)* API文档。由于API文档可能非常庞大（例如三星SmartThings的文档有9.3MB [35]），IoTGPT采用了检索增强生成流水线，该流水线会解析API文档并仅检索与每个子任务相关的部分。

在生成低级命令时，分配适当的参数值至关重要。当这些值在指令中被明确指定时，它们可以直接映射到相应的参数。然而，用户指令常常模糊不清且缺乏细节。持续向用户询问缺失信息会产生过多的交互开销。相反，IoTGPT将值保留为抽象占位符，这些占位符稍后会在精炼阶段根据用户偏好和设备上下文进行填充。

延续之前的例子，子任务“调节空调温度”可能会被转换为以下命令序列：[打开空调 → 设置模式为[模式值] → 设置温度为[温度值]]。由于原始指令中未指定参数值，它们被表示为抽象占位符。

```
{ "subtask": "调节空调温度", "commands": { { "desc": "打开空调", "device": { "name": "空调", "capability": { "name": "switch", "command": "on", "value": {} } }, { "desc": "设置模式为 [模式_值]", "device": { "name": "空调", "capability": { "name": "airConditionerMode", "command": "setAirConditionerMode", "value": { "string": "[模式_值]" } }, { "desc": "设置温度为 [温度_值]", "device": { "name": "空调", "capability": { "name": "thermostatCoolingSetpoint", "command": "setCoolingSetpoint", "value": { "decimal": "[温度_值]" } } } }
```

精炼阶段。精炼阶段的目标是使低级命令个性化，以适应用户偏好。为此，IoTGPT会向大语言模型提供*i)* 子任务及其

低级命令，以及*ii)*一份结构化的用户偏好摘要。这些偏好是从过去的设备交互中提取的，并以环境属性（例如温度、亮度、噪音）的级别表示。基于这些偏好，抽象命令的参数值被优化。例如，像[设置温度到 [温度值]] 这样的命令中的参数槽会被填入个性化值（例如20°C），或者在缺乏上下文特定偏好时使用默认值。关于此个性化过程的更详细描述见第六节。

B. 两步校正流程

即使采用了结构化的流水线，IoTGPT仍可能产生不完美的输出。首先，由于大语言模型的随机性，生成的指令可能表现出不一致或错误的行为（例如，API级错误）。其次，个性化可能未被充分指定：对于给定的上下文，系统可能缺乏足够的偏好证据，导致保守的默认设置或不匹配的参数选择。为了解决这两个问题，IoTGPT采用了一个两步校正流程，将自动化验证与可选的用户干预相结合。

自动自我校正。IoTGPT首先尝试使用一个由模拟设备组成的虚拟智能家居环境进行自动自我校正，该环境镜像了真实部署。在将指令应用到物理环境之前，IoTGPT会在虚拟环境中执行它们，并检查生成的执行日志中是否存在API违规和运行时故障。当检测到错误时，IoTGPT将错误信息反馈给大语言模型，并提示其仅修改命令序列中有问题的部分。这种基于日志的修订会重复进行，直到达到重试限制（例如，3–5轮），从而在无需用户参与的情况下提供轻量级的保障。

人在回路校正。如果在自我校正后错误仍然存在，或者如果用户想要覆盖模糊的个性化选择，IoTGPT会升级到交互式校正步骤。大语言模型会简要解释生成的指令，允许用户通过自然语言反馈进行干预。用户可以指定校正以修复剩余的无效性问题（例如，移除错误的子任务），或者通过调整参数或添加新的子任务来进一步个性化自动化。在后一种情况下，IoTGPT会从推导阶段重新调用流水线，以整合新的需求。此反馈在修正当前任务的同时，会更新系统的任务记忆和偏好配置文件，以供未来指令使用。

V. 分层任务记忆

由三阶段流水线产生的任务知识被系统地归档在IoTGPT的任务记忆中。该记忆作为一个知识库，使IoTGPT能够通过复用先前学习到的子任务和设备指令，让后续的物联网指令生成更加可靠和高效。在本节中，我们将描述IoTGPT如何组织和存储流水线的输出结果，以及这些知识如何被调用于未来的物联网指令生成。

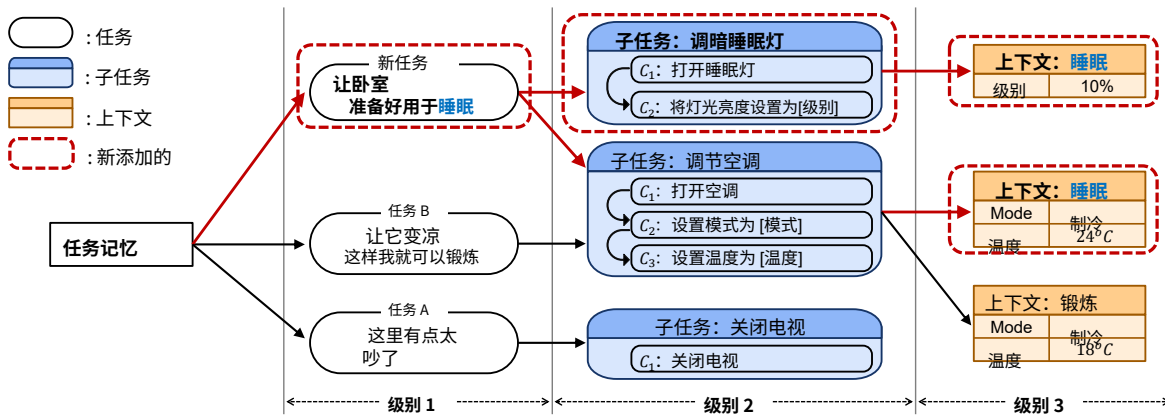


图 3. IoTGPT的任务记忆结构

A. 任务记忆结构

如图3所示，IoTGPT将其任务记忆组织为一个包含三个深度层级的分层有向无环图（DAG）：任务（级别1）、子任务（级别2）和上下文（级别3）。每个层级以不同的粒度捕获知识，从而实现复用和个性化。

任务级知识。每个任务节点对应一条以自然语言发出的用户指令。当新指令到达时，IoTGPT 首先会——在执行分解阶段之前——通过基于文本嵌入的余弦相似度比较新指令与已存储的任务，来检查记忆中是否已存在相似任务。如果存在相似任务，则可以复用已存储的知识（如稍后在回忆过程中所述）；否则，将创建一个新的任务节点并执行分解阶段。

子任务级知识。子任务节点在分解阶段创建，代表从任务中提取出的子任务操作。子任务节点与其源任务节点相连接。重要的是，子任务并不局限于单个任务，而是可以在多个任务间共享。例如，当从分解阶段生成一个新的子任务时，IoTGPT 会检查其与记忆中存在子任务节点的相似性；如果找到匹配项，它只需在当前任务节点与匹配的子任务节点之间添加一条新边。通过这种方式，先前存储的子任务可以直接在不同的任务中复用。如果未找到匹配项，则创建一个新的子任务节点。

每个子任务节点存储着在推导阶段生成的底层设备指令序列。当指令中明确指定了参数值，并且这些值在生成的指令中表现为具体数值时，IoTGPT 会先将它们替换为抽象占位符（例如，[温度值]），然后再将指令存储到任务记忆中。这种泛化操作保留了命令序列的结构，同时允许参数槽在后续被填充。

上下文层级知识：在精炼阶段添加上下文节点，用以捕捉命令参数在不同指令上下文（即指令的目标或意图）下是如何被个性化的。例如，在处理指令“让卧室准备好睡觉”时，IoTGPT会利用大语言模型提取关键词“睡眠”，该关键词代表了指令上下文，并创建一个相应的节点。

该节点存储了个性化参数值（例如，[模式值] = “制冷”，[温度值] = “20°C”）。如果已存在相同关键词的节点，IoTGPT 可以直接复用存储的值。

此外，来自人在回路校正的更新也会反映在任务记忆中。当用户添加/删除子任务或微调参数时，相应的子任务或上下文节点也会随之更新。因此，校正不仅应用于当前执行，还会为未来重用而积累，使得任务记忆最终保存的是经过用户验证的知识。

B. 任务知识回忆

在回忆任务知识时，IoTGPT 必须考虑到环境或用户上下文可能与知识最初被记忆时有所不同。例如，某些设备可能因故障而不再可用，或者相同的子任务可能出现在不同的指令上下文中。因此，根据具体情况确定知识应被重用到哪个深度层级至关重要。当新指令到达时，IoTGPT 按以下步骤进行：

任务/子任务级复用。如果记忆中存在匹配的任务，或者获取到的某些子任务与存储的子任务匹配，IoTGPT 会复用相应的子任务知识。在这种情况下，可以跳过该任务的分解阶段或那些子任务的推导阶段。然而，如果与子任务关联的某些设备不可用（例如，由于故障），则这些子任务会被排除。IoTGPT 随后会通知用户，并通过大语言模型建议替代子任务。

上下文级复用：对于可复用的子任务，IoTGPT 会通过比较上下文关键词来检查是否也能回忆起相关的上下文知识。如果存在匹配的上下文节点，系统会将子任务节点中的抽象低级命令与上下文节点中的参数值结合，从而避免冗余个性化。否则，IoTGPT 仅复用深度层级2的子任务级知识，保留抽象参数槽。这些参数槽随后会在精炼阶段通过个性化来填充。

通过利用此记忆召回机制，IoTGPT 减少了对重复大语言模型推理的依赖，实现了更一致、更高效的物联网指令生成。

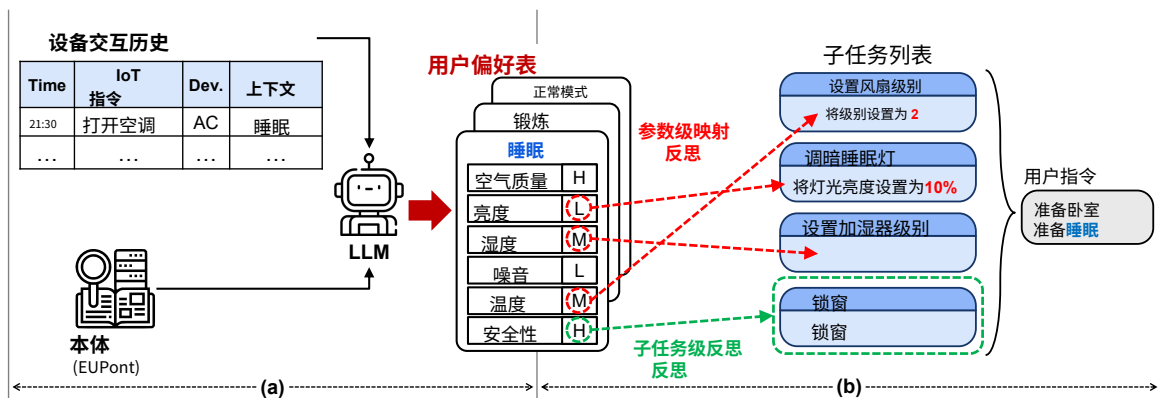


图 4. IoTGPT的自适应个性化

六、自适应个性化

个性化对于智能家居自动化不可或缺，因为即使是相同的子任务，也可能需要根据用户的偏好和上下文使用不同的参数值。然而，传统方法通常依赖于特定设备的交互历史，这使得在没有先前使用数据的情况下，很难将个性化扩展到其他设备。为了克服这一限制，IoTGPT 将用户偏好抽象为与设备无关的环境属性（例如，温度、亮度）。这种高级表示形式使得偏好能够在异构设备间灵活迁移。本节描述IoTGPT 如何根据环境属性提取用户偏好，并在精炼阶段应用这些偏好来个性化物联网指令。

A. 用户偏好提取

如图4(a)所示，IoTGPT 通过向大语言模型提供*i)* 设备交互历史和*ii)* 物联网指令与环境属性之间的关系，来提取高层次的用户偏好表示。为了捕捉这些关系，我们利用了EUPont本体 [36]，它定义了影响智能家居环境的六种环境属性：空气质量、亮度、湿度、噪音、温度 and 安全性。具体而言，该本体规定了每条物联网指令会影响哪些属性，以及这些属性是增加还是减少。例如，调节空调温度的主要效果是降低温度。通过提供此本体知识作为指导，IoTGPT 使大语言模型能够推断设备历史中物联网指令的效果。然而，偏好提取引发了几个问题：

上下文相关偏好。用户偏好通常随上下文而变化。例如，用户在睡眠时偏好的温度可能与运动时的偏好有显著差异。为了捕捉这种变化，我们的设备交互历史不仅包含原始设备指令，还包括指令上下文（即指令的目标或意图）。这使得大语言模型能够在不同的上下文（例如睡眠、运动或正常模式）下提取独立的偏好。此处，正常模式指不区分上下文的整体偏好。

定义属性等级。从过往物联网指令推断出的偏好，必须表达为每个环境

属性的等级。一种直接的方法是将设备参数范围划分为离散的级别（例如，低、中等、高），使用简单的基于规则的启发式方法。然而，为划分定义适当的规则具有挑战性，因为异构设备暴露的参数范围多种多样（例如，温度单位为°C，湿度单位为%，亮度单位为勒克斯），且离散的级别边界可能因属性而异。此外，对于安全性这类属性，等级并非由参数值决定，而是由更广泛的使用模式决定，例如锁窗频率或家用摄像头使用时长。手工制定的规则无法捕捉这种多样性和上下文依赖性。为了解决这个问题，IoTGPT利用大语言模型进行少样本提示，提供属性-等级映射的示例（例如，温度、湿度、安全性）。这使得大语言模型能够灵活地将参数范围划分或将使用模式解释为适当的属性等级。

通过此过程，IoTGPT会构建表格，将环境属性映射到偏好级别（例如，温度 → 低）。这些表格针对不同的上下文（例如，睡眠、运动）分别维护，使IoTGPT 能够捕捉用户偏好在不同情境下的细粒度变化。为使这些映射保持最新，提取过程会定期（例如，每日）运行，或在偏好信息过时、进入精炼阶段时按需运行。

B. 用户偏好反射

如图4(b)所示，在精炼阶段，IoTGPT会向大语言模型提供三个输入：*i)* 指令，*ii)*子任务，以及*iii)* 一个根据上下文选择的偏好表。为了选择该表，IoTGPT 会检查指令的上下文关键词是否与任何偏好表中定义的关键词匹配。如果找到匹配项，则使用相应的表；否则，系统将默认使用正常模式的上下文偏好。接着，IoTGPT 通过两种方式反映这些偏好：**参数级映射**。在此级别，大语言模型通过参考选定的偏好表来调整给定子任务的参数值。例如，在一个睡眠环境场景中，如果只有风扇可用而没有空调，IoTGPT仍然可以调整风扇的风速，以匹配表中指定的偏好温度水平，即使该风扇没有先前的使用历史。如第五节所述，这些个性化参数值随后会被存储在任务记忆的上下文节点中。

子任务级反思。除了调整参数，IoTGPT 还会根据用户偏好引入额外的子任务。例如，如果用户表现出对安全性属性的强烈偏好，大语言模型可能会添加一个新的子任务，例如“锁窗”，这是在分解阶段未曾考虑过的。这个新的子任务随后会进入推导阶段以生成其指令，并在需要时进行参数级映射。通过这种方式，偏好反射对分解阶段进行了补充，注入了原本可能被忽略的用户特定调整。

值得注意的是，当没有与指令匹配的上下文特定偏好表时，IoTGPT 会回退到正常偏好作为基线。然而，随着时间的推移，用户可以通过IoTGPT的人在回路校正功能，在不同上下文中微调参数值。这些调整会产生额外的交互日志，这些日志被输入到偏好提取过程中。该过程随后会更新现有表格或创建新的上下文特定表格，使它们能够逐渐偏离正常上下文。

七、实现说明

我们实现了一个IoTGPT的原型，它由一个安卓移动应用和一个通过Wi-Fi连接的后端服务器组成。该移动应用作为面向用户的界面：它接收自然语言指令并将其转发给后端。后端管理任务记忆并执行三阶段推理流程（分解-推导-优化）来处理每条指令。对于推理，我们为IoTGPT设计了一个与骨干大语言模型的模块化接口。默认情况下，我们的原型使用GPT-4o [37]，但骨干模型可以替换为其他大语言模型，而无需更改系统的其余部分。我们还将解码温度设置为0.0，以确保输出的一致性和确定性。对于任务知识检索，我们使用text-embedding-3-small模型 [38]，实现了一种基于文本嵌入的相似性搜索，通过计算存储节点与当前输入之间的余弦相似度来识别可复用的子任务或上下文。最后，后端通过三星SmartThings API [35]，与智能家居环境交互，同时物联网通信层被模块化，以便轻松地将IoTGPT扩展到其他生态系统，如Google Home或Apple HomeKit。我们的原型部署在一台Google Pixel 6智能手机上作为客户端设备，后端服务器配备英特尔i7-13700F CPU和32 GB内存。

VIII. 绩效评估

我们对IoTGPT进行了定量评估，以衡量其将自然语言指令翻译成物联网指令的能力。我们将IoTGPT与现有的基于大语言模型的智能家居代理在准确率、延迟和成本方面进行了比较。为了能在多样化的指令空间上进行受控测试，我们使用SmartThings Edge Driver [39]，构建了一个虚拟智能家居，模拟了涵盖照明、供暖和空调等类别的26种不同设备。

A. 对比研究

1) 实验设置：在对比研究中，我们按如下方式配置了实验环境：

基线。我们将IoTGPT与两种基于大语言模型的智能家居代理进行了比较：Sasha [8] 和SAGE [9]。Sasha是最早的基于大语言模型的智能家居代理之一，它采用了推理流水线，但将每条指令作为单一步骤执行，不进行任务分解。SAGE是我们实验时最先进的框架，采用了一种动态方法，由大语言模型在运行时规划推理模块的执行顺序。虽然这允许灵活分解，但SAGE不会记忆先前执行的知识；相反，它为每条指令从头开始重新规划。我们使用了SAGE的开源版本 [40]。由于Sasha的源代码不可用，我们根据其原始论文中详述的流水线架构重新实现了它。

数据集。我们使用了来自Sasha和SAGE的数据集。这些数据集中的每个任务都由一条自然语言指令和由人工标注者标记的真实设备指令配对组成。我们排除了与SmartThings平台不兼容的任务。此外，由于原始数据集包含的触发-动作案例相对较少，我们创建了15个额外任务。因此，我们的最终数据集包含97个任务：35个来自Sasha，47个来自SAGE，以及15个由我们新添加的。这些任务涵盖三个类别：55个直接控制命令，32个触发-动作规则，以及10个设备查询。

指标。为了严格评估准确率，我们定义了四个互补的指标。任务成功率 (*STR*) 是指所有必需的真实指令均被正确生成并执行的任务所占比例（忽略参数个性化）。过度控制率 (*ECR*) 是指包含真实情况中不存在的额外指令的任务所占比例，而控制不足率 (*ICR*) 是指至少缺少一项必需指令的任务所占比例。语法错误率 (*SER*) 是指包含至少一条违反SmartThings API规范（例如，语法错误）的指令的任务所占比例。请注意，ECR、ICR和SER并非互斥，因为一个任务可能同时包含不必要的指令、遗漏必需的指令以及包含语法错误。在效率方面，我们还测量了延迟

（从指令发出到虚拟智能家居中指令执行完成的端到端时间）和成本（根据所有模型调用中聚合的输入/输出令牌数，使用API提供商的定价计算出的每条指令的美元成本）。

流程。我们分两个阶段评估了IoTGPT：IoTGPT-Cold和IoTGPT-Warm。在IoTGPT-Cold中，智能体首次处理所有指令，同时构建任务记忆，并在知识可用时立即复用。随后的IoTGPT-Warm阶段则利用已完全填充的任务记忆重新处理相同的任务，但使用数据集的重新表述版本，以避免依赖简单的字符串匹配。具体来说，我们使用GPT-4o重新表述了自然语言指令，同时保留了语义意图。这迫使系统基于语义理解来利用知识。相比之下，Sasha和SAGE仅处理每条指令一次，没有阶段区分。为确保公平比较，我们禁

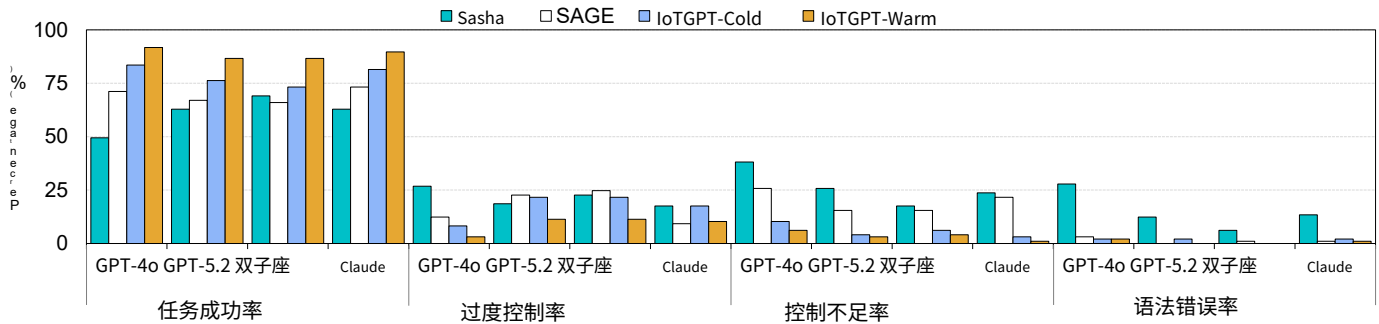


图 5. Sasha、SAGE 和 IoTGPT 的平均准确率

用了所有用户反馈步骤，并在每次运行中对所有方法应用相同的骨干模型。此外，为验证泛化能力，我们使用四种不同的大语言模型（GPT-4o [37], GPT-5.2 [41], Gemini 3 Flash [42], 和 Claude Opus 4.5 [43]）进行了评估，并对三次运行的结果取平均值。

2) 准确率：图 5 展示了三种方法在四种大语言模型骨干网络上的四项准确率指标。除了 Gemini 3 Flash, Sasha 在所有模型上都持续显示出最低的 STR（例如，在 GPT-4o 上为 49.48%）。因为它以整体方式处理指令，没有进行细粒度分解，Sasha 经常遗漏必要的设备指令，并在模糊或复杂任务下产生无效语法，导致其在所有智能体中具有最高的 ICR 和 SER。SAGE 通过利用更高级的推理模块并动态规划其执行顺序，通常比 Sasha 提高了 STR。然而，它对大语言模型的依赖常常产生冗长的规划链，这会因模型的随机性而增加错误累积。因此，SAGE 仍然表现出相对较高的 ICR，一些必需的指令被遗漏，任务也未完成。

相比之下，IoTGPT-Cold 实现了比 Sasha 和 SAGE 都更高的 STR，在四种骨干网络上最高达到 83.51%。它还产生了更低的 ICR 和 SER，表明遗漏的指令和语法错误更少。然而，IoTGPT-Cold 在所有模型上都显示出不可忽视的 ECR，范围从 8.25% 到 21.65%。我们的检查发现，许多这些“多余”的指令并非有害错误，而是潜在有用的补充。例如，在处理任务“如果家里没人，请节约能源”时，IoTGPT-Cold 不仅生成了关闭电器的真实指令，还额外生成了一个门锁指令。虽然该指令未包含在真实情况中，但一些用户可能认为它对于完成任务至关重要。此类输出在我们严格的 ECR 定义下会受到惩罚，但它们凸显了 IoTGPT-Cold 为满足用户需求提供更丰富支持的潜力。

关键的是，IoTGPT 的准确率在 IoTGPT-Warm 阶段得到进一步提升，此阶段利用了任务记忆来复用先前习得的知识。具体而言，与 IoTGPT-Cold 相比，在所有评估模型中，STR 最高提升了 18.31%，这直接归因于减少了对重复大语言模型推理的依赖。这些结果表明，随着任务记忆的增长，IoTGPT 可以通过积累可复用知识而持续改进。

有趣的是，我们观察到不同系统之间存在明显的骨干模型敏感性。由于 Sasha 依赖于相对简单的

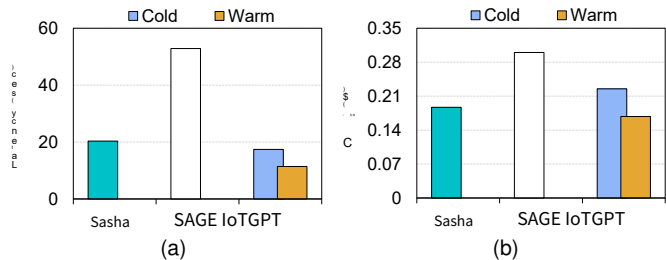


图 6. (a) Sasha、SAGE 和 IoTGPT 的平均延迟。(b) Sasha、SAGE 和 IoTGPT 的平均成本。

提示设计，其绩效对骨干模型的能力高度敏感。例如，在 Gemini 3 Flash 下，Sasha 在 STR 方面优于 SAGE，这主要是因为增强的推理能力大幅减少了语法错误。然而，尽管有此提升，Sasha 仍然记录了最高的 ICR，这表明单体推理仍然容易遗漏所需的指令。相比之下，IoTGPT 在使用更强大的模型（如 GPT-5.2 和 Gemini 3 Flash）时，其 STR 出现了反直觉的下降。矛盾的是，这源于它们的高级推理能力，这种能力倾向于生成更多额外的、上下文感知的子任务；虽然这些新增内容可能有用，但它们并未包含在固定的真实情况中，从而抬高了 ECR 并降低了 STR。

3) 延迟与成本：图 6 展示了使用 GPT-4o 骨干网络时，每种方法每条指令的平均延迟和成本。SAGE 在这两项指标上均表现最差，这是因为它严重依赖大语言模型来动态规划推理模块。虽然这种灵活性有时能提升推理质量，但也带来了巨大的开销，尤其是在规划链变得过长的复杂任务中。事实上，我们观察到有些指令耗时长达 151 秒，单条指令成本高达 4.2 美元（例如“如果家里没人，就节约能源”）。相比之下，Sasha 的延迟和成本相对 SAGE 较低，这主要得益于其简单的单体流水线，它直接处理指令而无需分解。然而，这种简单性是以牺牲准确率为代价的，正如我们的准确率结果所示。

与此同时，IoTGPT-Cold 呈现出一个有趣的权衡。尽管由于其多阶段流水线的累积令牌使用量，它比 Sasha 产生了更高的成本，但它实现了更低的延迟。我们将这一反直觉的结果归因于在每个阶段使用了更结构化、更详细的提示词，这可以减少大语言模型的推理时间，尽管存在多个阶段。

此外，IoTGPT-暖 展现出显著的效率提

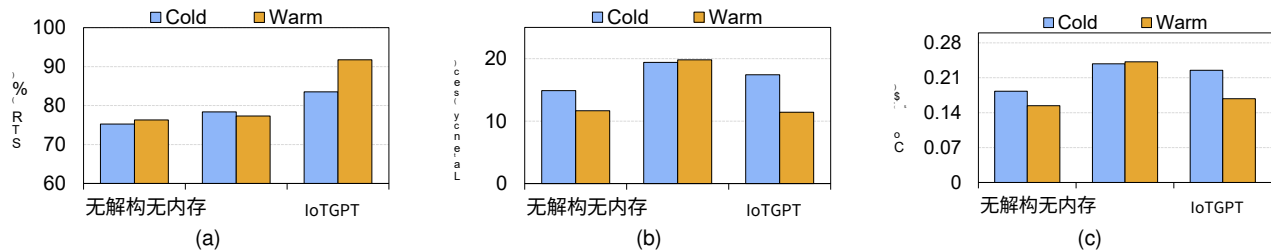


图 7. IoTGPT 在冷启动 (Cold) 与热启动 (Warm) 设置下的消融研究结果

通过利用任务记忆实现了效率提升, 在所有方法中获得了最低的延迟和成本。与IoTGPT-Cold相比, 延迟和成本分别降低了34.44%和25.33%。这种降低源于复用已学习的子任务, 而无需重复进行大语言模型推理。总体而言, 这些结果表明, 随着其任务记忆积累可复用知识, IoTGPT 的效率会逐步提高。

B. 消融研究

我们进行了一项消融研究, 以分析 IoTGPT 关键特性的贡献。具体来说, 我们移除了任务分解和任务记忆, 以评估它们对准确率、延迟和成本的影响。请注意, 个性化未包含在此消融研究中, 因为它依赖于主观用户体验, 并在第 IX-A 节通过用户研究单独评估。

1) 实验设置: 对于消融研究, 我们按如下方式配置实验环境:

基线。我们将 IoTGPT 与两个自定义基线进行了比较:

NoDecomp 和 NoMem。NoDecomp 移除了任务分解, 并直接从用户指令推断所需的设备指令, 遵循先前智能体

[8], [10], [11] 中使用的仅推导设计。NoMem 保留了任务分解, 但禁用了任务记忆, 在执行流水线时不复用过往子任务的知识。为确保公平比较, 两个基线均使用与 IoTGPT 相同的提示词。

数据集与指标。我们使用了与比较研究中相同的数据集, 并将 STR 作为主要的准确率指标进行衡量, 同时考量延迟和成本。

流程。我们在冷启动和热启动阶段评估消融研究, 并采用 GPT-4o 作为所有智能体的骨干模型。在冷启动阶段, 每个智能体首次处理所有指令。在此阶段, IoTGPT 和 NoDecomp 都构建了任务记忆并即时复用可用知识。关键区别在于粒度: IoTGPT 在细粒度子任务级别存储知识, 而 NoDecomp 由于缺乏分解, 仅能在任务级别存储知识。在热启动阶段, 每个智能体使用数据集的改写版本重新处理相同的任务, 这与比较研究中的做法一致。IoTGPT 和 NoDecomp 利用了它们已构建的记忆, 而 NoMem 则针对每条指令从头开始执行推理流水线。

2) 准确率: 图7(a)展示了每种方法的STR。我们通过比较IoTGPT 与NoDecomp来分析任务分解的影响。在冷启动阶段, IoTGPT的表现优于NoDecomp, 这表明将任务分解为子任务能提高推理准确率。这一性能差距在

热启动阶段进一步扩大。这主要是因为NoDecomp 依赖于粗粒度任务级记忆, 当重述的指令表现出高语言变体时, 常常无法检索到可复用知识, 仅带来1.37%的边际STR提升。相比之下, IoTGPT利用了细粒度子任务级复用; 即使任务级匹配因指令重述而失败, 它也能复用为分解出的子任务所学的知识。这种细粒度复用使得IoTGPT能够在不同任务共享共同推导出的子任务时, 从已学知识中获益, 从而实现了9.87%的显著提升。这意味着任务分解不仅能提升推理性能, 还能通过与任务记忆协同, 在语言变体下实现稳健的复用。

接下来, 我们通过比较IoTGPT 与NoMem来考察任务记忆的作用。在冷启动期间, IoTGPT 从成功执行中构建任务记忆, 并复用可用知识, 而不是反复依赖大语言模型, 从而减少了指令生成出错的机会。相反, 完全依赖大语言模型且没有记忆的NoMem, 由于频繁出错, 表现出较低的STR。值得注意的是, 在热启动阶段, NoMem在改写数据集上的STR下降了1.31%, 尽管它重新处理的是语义上与其之前见过的任务相似的任务。这一观察突显了任务记忆在减少指令生成错误方面的重要作用。

3) 延迟与成本: 图7(b)和7(c)分别展示了每种方法的延迟和成本。将IoTGPT与NoDecomp进行比较时, IoTGPT 在冷启动阶段会产生更高的开销。这一增加主要源于任务分解所需的提示中包含了详细的设备规格, 这是为了提升准确率而做出的必要权衡。然而, 在热启动阶段, IoTGPT通过利用任务记忆来绕过冗余的大语言模型推理, 实现了与NoDecomp 相当的效率。相比之下, NoMem在两个阶段都持续表现出最高的延迟和成本, 因为它被迫为每条指令重复分解过程。这些结果表明, 虽然任务分解会带来固有的开销, 但任务记忆能显著减轻这一负担, 从而确保系统效率。

IX. 用户研究

为补充第八节中的定量评估, 我们进行了两项用户研究: i) 研究1: 变化智能家居环境下的个性化, 该研究检验了IoTGPT 能否在设备配置变化时适应用户偏好; 以及 ii) 研究2: 跨任务难度的整体可用性

表 I 用于个性化研究的基础与新智能家居环境的设备配置。

	基础环境	新环境
温度	空调, 恒温器, 温湿度传感器	温湿度传感器, Fan
湿度	空调	Fan
亮度	光线, 百叶窗	光线, 百叶窗
噪音	TV, 吸尘器	TV, 吸尘器
空气质量	空调, 空气质量传感器	智能窗户, 空气质量传感器
安全性	门锁	家用摄像头

级别, 该研究评估了与基线系统相比, IoTGPT 在不同复杂度的自动化任务中的可用性。

我们通过在线社区广告招募了15名参与者(11名男性, 4名女性; 平均年龄= 28.1岁, 年龄范围 = 24–32岁)。其中4名参与者有使用物联网平台(如Google Home或三星SmartThings)的经验, 14名参与者有使用对话式智能体(包括Google Assistant、苹果Siri或三星Bixby)的经验。每次会话持续60–80分钟, 参与者获得10美元报酬。此项用户研究符合我们机构的机构审查委员会政策及知情同意书规定。

A. 研究1: 自适应个性化评估 *ion*

我们进行了一项用户研究, 以评估IoTGPT的个性化能力。与量化指标不同, 生成的参数是否反映用户偏好本质上是主观的。因此, 我们依靠人工评估来检验IoTGPT 是否能在设备可用性变化时有效地对参数进行个性化调整。

1) 实验设置: 我们按如下方式搭建实验环境:

智能家居设置。如表I所示, 我们准备了两个环境: BASE和NEW, 每个环境包含九台设备。在EUPont本体下, 每台设备都与特定的环境属性相关联。虽然两个环境共享部分设备, 但它们在部分设备上存在差异。例如, BASE包含一台空调、一个恒温器和一个智能门锁, 而NEW则包含一台风扇、一扇智能窗户和一个家用摄像头。此设置模拟了异构的智能家居环境, 挑战系统根据可用设备能力适应用户偏好的能力。

待比较的方法。我们比较了与先前实验相同的方法, 即Sasha 和SAGE。对于IoTGPT, 我们没有分别评估其在冷启动和热启动下的表现, 因为任务记忆的存在与否对个性化质量没有显著影响。我们使用GPT-4o作为所有方法的骨干模型。

数据集。如表II所示, 我们从第八节介绍的数据集中选取了8个任务。任务的选择基于个性化难度, 该难度由需要考虑的环境属性数量定义: 涉及的属性越多, 难度越高。

流程。为了启动个性化, IoTGPT需要设备交互日志来构建偏好表。然而, 由于在有限的实验环节中从每位参与者收集此类日志实际上不可行, 我们

使用GPT-4o随机生成了100条交互日志。从这些日志中, IoTGPT提取了关于环境属性的偏好表, 并在研究期间使用它们。在实验开始前, 我们向参与者提供了研究中使用的偏好表的解释, 并在他们确认理解后才参与。为了进行比较, SAGE 和Sasha被提供了相同的交互日志, 但处理方式不同。SAGE 完全依赖这些日志来推断设备特定的偏好, 这符合其原始设计; 而Sasha——原本缺乏个性化功能——则通过在其提示中包含此历史记录进行了扩展。

实验采用被试内设计。为了最小化顺序效应, 每个参与者看到的系统呈现顺序是随机的。首先, 在每个会话中, 所有系统都在BASE环境下从头处理8个任务。然后, 它们在设备配置修改后的NEW 环境中重新执行相同的任务。每个阶段结束后, 参与者使用七点李克特量表(见表III)从三个方面对输出结果进行评分: 感知准确性(Q1)、设备级个性化(Q2)和参数级个性化(Q3)。Q1反映了参与者对正确性的主观感知, 与真实情况无关, 因此是对第八节报告的客观准确性的补充。Q2和Q3评估了每个系统在设备和参数级别上根据用户偏好调整输出的有效性。系统身份被隐藏以防止偏见, 并通过后续访谈捕捉参与者的推理过程。

2) 结果: 表III总结了参与者对三个系统在BASE 和NEW环境下的评分。在Q1(感知准确性)方面, IoTGPT在两种环境中都获得了最高评分, 其次是SAGE和Sasha。这一趋势与第八节中的客观准确性结果一致, 表明参与者同样认为IoTGPT 是最准确的系统。例如, P2指出IoTGPT “清晰地列出了完成任务所需的所有指令”, 而P11则批评Sasha 产生了“许多毫无意义且准确率极低的输出”。

在Q2和Q3中, 参与者一致给予IoTGPT 最高评价, 强调其“在两种环境中都尽可能反映了用户偏好”(P4)。相比之下, SAGE被认为仅部分有效: P12观察到它“设备选择相当合理, 但参数值感觉相差甚远”, 并进一步指出“在NEW 环境中, 它似乎只满足了部分多重偏好”。Sasha评分最低, P15评论说它“忽略了用户偏好, 而是自行判断应提供何种服务”。这些结果突显了IoTGPT即使在设备配置变化的情况下, 也始终能提供最强的个性化。

为了检验三种方法在Q1、Q2和Q3得分上的统计显著性, 我们进行了弗里德曼检验。在BASE中, 所有三个问题均观察到显著差异(Q1: $\chi^2(2) = 24.03, p < 0.001$, Q2: $\chi^2(2) = 24.4, p < 0.001$, Q3: $\chi^2(2) = 24.4, p < 0.001$)。为了探究成对差异, 我们进行了事后威尔科克森符号秩检验, 并采用霍尔姆-邦费罗尼校正。分析显示, 所有方法对之间均存在显著差异(全部 $p < 0.05$)。类似地, 在NEW中, 所有三个问题也发现了显著差异(Q1: $\chi^2(2) = 24.4, p < 0.001$, Q2: $\chi^2(2) = 24.4, p < 0.001$, Q3: $\chi^2(2) = 24.4$,

表II 个性化研究中使用的任务集。任务根据涉及的环境属性数量分为三个难度等级。B = 亮度, T = 温度, H = 湿度, N = 噪音, A = 空气质量, S = 安全性。

难度等级 (属性数量)	任务 (用户指令)	环境属性					
		B	T	H	N	A	S
简单 (1-2)	帮我冷静一下		✓				
	改变心情, 让我振作起来 这里有点太吵了	✓			✓ ✓		
中等 (3-4)	设置一个良好的学习环境	✓	✓	✓	✓		
	将房间设置为轻松的氛围	✓	✓		✓		
	我累了——让它变得舒适, 这样我就可以休息了	✓	✓		✓		
困难 (5-6)	让房子充满舒适的氛围	✓	✓	✓	✓	✓	
	为我创造最佳的睡眠环境	✓	✓	✓	✓	✓	✓

表三 研究一的调查结果。每项评分均在任务完成后使用7点李克特量表收集。数值代表平均值, 标准差显示在括号中。

	基础环境			新环境		
	Sasha	SAGE	IoTGPT	Sasha	SAGE	IoTGPT
Q1. 是否包含了所有必要的设备指令 (无论偏好如何)?	2.47 ± 0.21	3.68 ± 0.13	5.95 ± 0.05	1.93 ± 0.19	3.90 ± 0.13	5.78 ± 0.05
Q2. 所选的设备是否适合 偏好?	2.44 ± 0.23	3.52 ± 0.14	6.28 ± 0.04	1.93 ± 0.19	3.72 ± 0.14	5.63 ± 0.09
Q3. 设备设置是否根据 个人偏好进行了调整?	2.20 ± 0.21	3.04 ± 0.11	6.02 ± 0.05	1.93 ± 0.19	3.18 ± 0.14	5.57 ± 0.10

$p < 0.001$)。事后威尔科克森符号秩检验揭示了所有方法对之间均存在显著差异 (全部 $p < 0.05$) , 这表明IoTGPT 在偏好适应方面显著优于两个基线系统。

B. 研究二：整体可用性评估

我们进行了一项用户研究, 以评估IoTGPT 在现实智能家居场景中的可用性和交互体验。我们将用户使用IoTGPT 的体验与基线方法进行了比较, 重点关注我们的设计选择是否能转化为切实的益处。具体来说, 我们试图回答两个研究问题: *i) IoTGPT*的分解流程是否能让用户实现更有效的自动化成果? 以及*ii) IoTGPT*的分层任务记忆是否能有效提供可靠且快速的响应?

1) 实验设置：我们按如下方式设置了实验环境：

智能家居设置。研究中使用的智能家居环境包含七种常见的物联网设备：一个存在传感器、光线、门锁、百叶窗、空调、冰箱和电视。

待比较的方法。我们将IoTGPT与两个基线进行了比较：SmartThings应用和SAGE。我们将SmartThings应用（在苹果iPad 4上测试）作为实际参考点, 以了解用户当前如何通过主流图形界面执行物联网指令。我们选择SAGE作为先前基于大语言模型的智能体中最强研究基线, 因为在我们的比较评估中, 它始终比Sasha 取得了更好的准确率和个性化效果。我们在两种条件下评估了IoTGPT , 以捕捉任务记忆对可用性的影响: IoTGPT-Cold, 它从空记忆开始, 代表首次用户体验; 以及IoTGPT-Warm, 它预加载了基于第VIII节中97个任务构建的任务记忆, 代表记忆积累后的持续使用。两种条件使用了相同的偏好

表, 这些表推导自第IX-A节中的人物设定, 因为为每位参与者收集交互历史是不切实际的。请注意, 我们使用GPT-4o作为所有智能体的骨干模型。

流程。在每个实验环节开始时, 每位参与者都会获得关于智能家居技术和对话式智能体的介绍。我们还解释了研究中使用的智能家居配置以及如何使用每种方法, 随后是一个简短的教程, 让参与者在执行主要任务前能先练习使用这些系统。

随后, 参与者被要求使用全部四种方法（SmartThings应用、SAGE、IoTGPT-Cold和IoTGPT-Warm）完成三个难度递增的任务场景（简单、中等和困难）。这些场景对所有参与者都是相同的。对于每个场景, 我们既提供了描述, 也提供了一个示例指令（见表IV）, 同时允许参与者以不同的方式表述他们的指令。所有任务都是触发-动作规则, 并按难度递增的顺序呈现。为了减轻不同方法间的顺序效应, 它们的顺序在参与者间进行了平衡。

在每个场景完成后, 参与者填写了一份NASA任务负荷指数量表以评估感知负荷, 并接受了即时反馈访谈。这一过程针对四种方法下的所有三个场景重复进行。最后, 在完成所有任务后, 参与者使用系统可用性量表（SUS）评估了四种方法的可用性。会议以一次访谈结束, 参与者在访谈中反思了每种方法的优缺点。

2) 结果：我们现在介绍第二次用户研究的结果, 该研究在指令生成时间、感知负荷和可用性方面, 比较了四种方法在不同难度任务上的表现。

任务完成时间。我们测量了在三种场景下完成每个自动化任务所需的时间。为了考虑不同的交互模式, 我们为每种方法定义了不同的度量标准：对于SmartThings应用,

表 IV 可用性研究的任务场景，按难度级别（简单、中等、困难）分组

难度级别	场景描述	示例指令
Easy (≥ 2 设备, 明确指定)	凌晨三点, 你口渴醒来, 悄悄走向厨房。你打开冰箱, 但房间太暗, 找不到一杯水。你心想, 如果每次打开冰箱门时餐厅灯都能自动亮起, 那就太好了。	“当我打开冰箱时, 打开餐厅的灯 当我打开冰箱时房间”
中等 (≥ 4 设备, 未指定)	每天晚上, 你都会以在电视上看电影来结束一天。为了更好地专注于观影体验, 你通常会布置好周围的环境。然而, 每次重复进行这种准备工作, 开始让你觉得越来越麻烦。	“当我看电视时, 我希望我的房子 能像电影院一样”
Hard (≥ 6 设备, 未指定)	在漫长而疲惫的一天结束后, 你走向卧室, 并为睡眠设置好环境。然而, 当你每晚重复同样的流程, 在一天结束时自己动手准备时, 开始让人感到越来越单调和烦人。	“让房间变得舒适, 以便 我晚上”睡觉

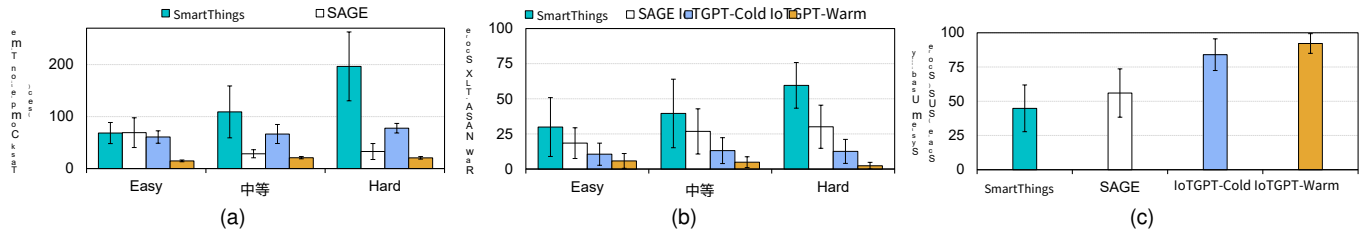


图8. (a) 各场景下的平均任务完成时间。(b) 各场景下的平均原始NASA任务负荷指数数量表得分。(c) 各方法的平均可用性得分。每个得分均在100分制上评估, 误差条显示标准误差。

它衡量的是从启动自动化设置到点击“保存”按钮（该按钮会在智能家居环境中应用已配置的自动化）的人机交互时间。对于对话式智能体（SAGE 和IoTGPT），它表示从参与者提交自然语言指令到生成的指令在智能家居环境中完全应用为止的端到端延迟。

如图8(a)所示, 除了简单场景外, SmartThings应用始终需要最长的任务完成时间。此外, 其完成时间随任务难度急剧增加: 在困难场景中, 时间比简单场景高出187.7%。这反映了手动指定每个设备操作所带来的开销, 随着任务变得更加复杂, 这种开销会急剧增加。与此同时, SAGE 在简单场景中耗时最长, 但在中等和困难场景中却呈现出时间减少的反常趋势。这一反直觉的结果源于 SAGE 将触发-动作任务误解为更简单的直接控制指令, 这虽然减少了任务完成时间, 却导致了不准确的自动化²。这凸显了过度依赖大语言模型的风险。

IoTGPT-Cold 在任务变得更复杂时, 所需时间也有所增加, 但其增长幅度比SmartThings平缓（从简单到困难任务, 时间增加28.1%）。相比之下, IoTGPT-Warm利用任务记忆, 在所有场景下都实现了显著更低的时间, 与IoTGPT-Cold 相比, 在简单、中等和困难场景下分别减少了75.67%、68.54%和73.46%的时间。这些结果凸显了复用已学习子任务所带来的效率提升。

我们采用重复测量方差分析来分析不同方法在任务完成时间上的差异, 并在事后检验中应用邦费罗尼校正以控制多重比较。方差分析显示, 方法对所有场景的完成时间均有显著影响

²具体而言, SAGE未能在中等和困难场景下为所有参与者任务生成正确输出, 而两种IoTGPT变体在所有场景下均能持续生成正确的物联网指令。

（简单: $F(3, 42) = 31.84, p < 0.001$; 中等: $F(3, 42) = 32.59, p < 0.001$; 困难: $F(3, 42) = 79.19, p < 0.001$ ）。

在简单场景中, 事后比较显示, IoTGPT-Warm的速度显著快于SmartThings应用、SAGE和IoTGPT-Cold（所有比较均为 $p < 0.001$ ）。其他成对比较均未达到显著差异。对于中等和困难场景, 我们将SAGE排除在统计比较之外, 因为它对任务的误解导致了无效输出。在中等场景中, IoTGPT-Warm的表现优于SmartThings应用和IoTGPT-Cold（所有比较均为 $p < 0.001$ ），而SmartThings应用与IoTGPT-Cold之间未发现显著差异。在困难场景中, 所有成对比较均具有显著性（所有比较均为 $p < 0.05$ ）。在所有场景中, IoTGPT-Warm始终产生最快的时间, 并且随着任务复杂度的增加, 其优势变得更加明显。

感知任务负荷。图8(b)展示了每种场景下的平均原始NASA任务负荷指数数量表得分。SmartThings应用始终产生最高的负荷, 其次是SAGE。SAGE的高分主要源于糟糕的任务结果, 因为它经常无法生成令人满意的指令, 导致绩效（表示更差的感知结果）和挫败感方面的评分更高。相比之下, IoTGPT-Cold 显著降低了负荷, 而IoTGPT-Warm 进一步将其降至所有场景中的最低水平。这表明IoTGPT有效地最小化了认知负担, 使用户能够管理复杂任务而不会感到不知所措。

弗里德曼检验显示, 在所有场景中, 方法均存在显著影响（简单: $\chi^2(3) = 18.69$; 中等: $\chi^2(3) = 26.49$; 困难: $\chi^2(3) = 37.54$; 所有 $p < 0.001$ ）。经霍尔姆-邦费罗尼校正的事后威尔科克森符号秩检验显示如下。在简单场景中, IoTGPT-暖的工作负荷显著低于SmartThings应用（ $p = 0.007$ ）和SAGE（ $p = 0.009$ ），而IoTGPT-冷与IoTGPT-暖之间的差异不显著。

($p = 0.129$)。IoTGPT-冷的工作负荷也显著低于SmartThings应用($p = 0.017$)，但与SAGE($p = 0.0905$)相比不显著。在中等场景中，IoTGPT-暖的工作负荷再次显著低于SmartThings应用($p = 0.005$)、SAGE($p < 0.001$)和IoTGPT-冷($p = 0.016$)。IoTGPT-冷的工作负荷也显著低于SmartThings应用($p = 0.002$)和SAGE($p = 0.025$)。在困难场景中，所有两两比较均显著(所有 $p < 0.05$)。

可用性。图8(c)展示了每种方法的平均系统可用性量表分数。SmartThings应用获得的评分最低，其次是SAGE，而两个版本的IoTGPT得分都显著更高。特别是，IoTGPT-Cold 已经达到了优秀范围，而IoTGPT-Warm 则进一步进入了最佳想象类别。这些结果突显了IoTGPT 在感知可用性上相对于两个基线的明显优势。

弗里德曼检验揭示了方法对可用性有显著影响($\chi^2(3) = 32.05, p < 0.001$)。事后威尔科克森符号秩检验证实，IoTGPT-Warm被认为比SmartThings应用($p < 0.001$)、SAGE($p = 0.003$)和IoTGPT-Cold($p = 0.037$)都显著更易用。IoTGPT-Cold 也显示出比SmartThings应用($p = 0.001$)和SAGE($p < 0.001$)都显著更高的可用性。

C. 总体关键启示

我们在两项用户研究中的发现，为智能家居自动化系统的设计带来了四项关键启示。

手动与对话式界面。参与者一致认为，随着任务复杂度的增加，SmartThings应用变得繁琐。例如，P3指出：

“现有平台感觉太复杂、太困难，因为有太多项目需要选择。”同样，P13评论道：“我认为这对年长用户来说会难以使用。”当面对更复杂的任务时，参与者在信心和清晰度上遇到了困难；P6承认：“我不确定我的设置是否正确，”而P5则表示：“我不知道该怎么做才能让房子有电影院的感觉。”总而言之，这些评论凸显了手动、可视化界面的可扩展性限制。

相比之下，对话式智能体被认为更自然、更高效，尤其对于复杂任务。参与者赞扬了IoTGPT：P1观察到，“仅用一个语音指令就能控制多个设备，非常直观和方便。”P7补充道，“系统能建议我自己都想不到的设备控制方式，这令人印象深刻。”另一位参与者强调了IoTGPT 如何处理模糊指令：P9反思道，“它能够自然地具体化并执行像‘让它有电影院的感觉’这样模糊的指令，这很棒。即使是我，也得仔细想想这对我来说到底意味着什么。”这些发现凸显了自然语言交互对于智能家居自动化的价值。

过度依赖大语言模型的风险。虽然现有基于大语言模型的智能体支持自然的交互，但它们对大语言模型的严重依赖导致了不稳定的结果和较慢的响应。SAGE尤其如此，它为每个任务从头重新规划推理模块，并反复查询大语言模型直至完成，这使其高度依赖大语言模型。这不仅延长了执行时间，也增加了出错的可能性。

当任务变得更加复杂或模糊时。参与者直接指出了这些缺点。P2评论道：“即使是简单的任务，花费的时间也比我预期的要长。”P10指出：“它忽略了我指令中的条件（触发器），只生成动作部分，这使得它在现实生活中不实用。”P8分享了一个更严重的担忧：“系统突然将电视音量调到了100。对于这种行为需要有一个安全防护机制。”总之，这些发现凸显了需要互补机制——例如任务记忆——来减少对大语言模型的重复依赖，并确保更可靠的绩效。

冷启动与热启动可用性。我们的研究结果表明，IoTGPT 即使在没有任何先前任务记忆的冷启动条件下，也提供了强大的可用性。参与者仍然能够以相对较低的工作负荷和较高的满意度完成复杂的自动化任务，这表明系统的分解流程即使在没有预先学习的情况下也能带来显著的益处。然而，热启动变体进一步放大了这些优势：通过复用累积的任务记忆，它减少了指令生成时间，最小化了工作负荷，并提升了整体可用性。这证明，虽然IoTGPT开箱即用，但其可用性会随着任务记忆的增长而逐步提升，为长期采用提供了清晰的路径。参与者的反馈强化了这些观察。P9指出：“即使没有预先学习，它仍然运行良好，感觉可用。”P12强调了渐进式的改进：“较早的版本（IoTGPT-Cold）花费的时间比我预期的要长，但下一个版本（IoTGPT-Warm）能快速给出想要的结果，这非常令人满意。”同样，P10补充道：“较早的版本已经足够好用，但学习后的版本速度快到足以处理即时指令，这使它感觉实用得多。”

个性化的重要性。我们的研究强调个性化是智能家居自动化的核心要求。参与者看重那些能根据其偏好和上下文定制指令的系统，并指出这种支持让自动化感觉更实用、更值得信赖。例如，P10解释说：“很方便的是，多个设备都根据给定的偏好进行了调整，而无需我手动配置它们。”同样，P7评论道：“系统似乎甚至能建议我自己都未曾意识到有用的设备使用模式。”除了静态个性化，研究1的结果指出了适应性的重要性：当设备配置发生变化时，在环境属性级别映射偏好的系统能更有效地保留用户偏好。正如P4评论的那样：“令人印象深刻的是，即使设备不同，系统仍然能反映我的偏好。”总之，这些发现强调，个性化——尤其是自适应个性化——对于在动态家居环境中维持可用性至关重要。

十、局限性与未来工作

用户偏好冲突困境。当用户在不同环境属性上的偏好发生冲突时，个性化可能变得模糊不清。例如，一位用户

既偏好低温又偏好低噪音，可能会面临一个困境：运行空调可以降低室温，但会带来噪音这一副作用。在这种情况下，系统应优先考虑温度还是噪音并不明确。由于我们基于EUPont本体构建的当前原型并未考虑此类副作用，因此默认会开启空调。然而，如果用户更看重安静而非温度，这可能导致次优的个性化结果。为解决此问题，未来的工作应扩展本体以表示设备副作用，并优化偏好粒度，从而使系统能更有效地解决此类冲突。

对本地推理的需求。虽然IoTGPT通过任务记忆降低了指令生成延迟，但一些参与者仍感知到延迟。例如，P4评论道：

“尽管执行时间得到了优化，但感觉仍然很长。”除了绩效之外，对数据隐私的担忧也出现了。正如P6所指出的：“自然语言指令可能无意中包含私人信息，如果由基于云的大语言模型处理，这些信息可能会被暴露。”这些观察结果表明，未来需要在实现本地推理方面开展工作。开发专为智能家居自动化定制的轻量级大语言模型，既可以减少延迟，又可以降低隐私风险，使此类系统在日常使用中更加实用和值得信赖。

上下文感知的局限性。IoTGPT的一个局限性在于，它仅从用户指令推断上下文，这无法完全捕捉情境因素。例如，将活动上下文（例如，通过视觉语言模型分析家用摄像头图像来识别用户是在休息还是在锻炼）与环境状态上下文（例如，传感器测量的温度、湿度或噪音）相结合，可以实现对物联网指令更准确的优化。然而，整合这种多模态上下文可能导致任务记忆过度碎片化，使得个性化知识的重用变得困难。因此，未来的工作应探索抽象技术——例如上下文的聚类或层次化表示——以捕捉多样化的情境，同时保持可重用性。

从人工检查到内置安全机制。在我们当前的雏形系统中，过滤不安全或幻觉指令的责任在于用户，用户必须审查生成的自动化规则，并在必要时提供纠正反馈。虽然这种方法防止了即时风险，但给用户带来了额外负担并降低了可用性。为解决这一局限，未来的工作应探索自动化安全机制，在指令执行前主动验证或调整指令。例如，如果系统错误地生成了将电视音量调至100的指令，内置安全机制可以自动将值限制在安全阈值内（例如50），而无需用户干预。此类验证和故障安全措施将提高可靠性，同时减轻用户负担，使IoTGPT能够在最小监督下提供安全的自动化。

长期用户体验。我们的评估仅限于短期使用，因为参与者在用户研究中与IoTGPT交互了大约60分钟。虽然这些会话为可用性和个性化提供了有用的见解，但它们无法完全捕捉用户在日常生活中如何长期适应或依赖IoTGPT。未来的工作应

开展纵向研究，以考察在现实世界智能家居环境中，持续使用如何影响用户信任、系统适应性以及整体满意度。

XI. 结论

我们介绍了IoTGPT，一个基于大语言模型的智能家居代理，它利用任务分解和任务记忆，以可靠、高效且个性化的方式生成物联网指令。IoTGPT将结构化推理与记忆重用相结合，以减少物联网指令生成中的不可靠性、延迟和成本，同时通过设备无关的偏好建模支持自适应个性化。我们在绩效基准测试和用户研究中的评估表明，与基线相比，其在准确率、效率和可用性方面均有提升。我们相信，IoTGPT代表了迈向更可靠、以用户为中心的人工智能代理的具体一步，其影响不仅限于智能家居，更延伸至可靠且自适应的自动化至关重要的更广泛场景。

参考文献

- [1] T. Thakur, “2025年物联网统计：设备、安全性与采用情况,” <https://sqmagazine.co.uk/internet-of-things-statistics/>, 2025, 访问日期: 2025-07-30.
- [2] L. Bainbridge, “自动化的讽刺,” *IFAC会议论文集*, vol. 15, no. 6, pp. 129–135, 1982.
- [3] 三星, “SmartThings,” <https://www.samsung.com/sec/smarththings/app/>, 2025, 访问日期: 2025-07-23.
- [4] 谷歌, “Google Home,” <https://home.google.com/welcome/>, 2025, 访问日期: 2025-07-23.
- [5] 苹果, “Apple Home,” <https://www.apple.com/kr/home-app/>, 2025, 访问日期: 2025-07-23.
- [6] 谷歌, “Hey Google,” <https://assistant.google.com/>, 2025, 访问日期: 2025-07-10.
- [7] 苹果, “在所有苹果设备上使用Siri,” <https://support.apple.com/en-us/105020>, 2025, 访问日期: 2025-07-10.
- [8] E. King, H. Yu, S. Lee, and C. Julien, “Sasha: 智能家居中基于大语言模型的创造性目标导向推理,” *ACM交互式、移动、可穿戴和普适技术汇刊*, vol. 8, no. 1, pp. 1–38, 2024.
- [9] D. Rivkin, F. Hogan, A. Feriani, A. Konar, A. Sigal, and X. Liu, “通过自主大语言模型代理实现的AIoT智能家居,” *IEEE物联网期刊*, vol. 12, no. 3, pp. 2458–2472, 2025.
- [10] Y. Shi, X. Liu, C. Yu, T. Yang, C. Gao, C. Liang, and Y. Shi, “弥合智能家居中自然用户表达与复杂自动化编程之间的鸿沟,” <https://arxiv.org/abs/2408.12687>, 2024, arXiv:2408.12687.
- [11] S. Gallo, F. Paternò, and A. Malizia, “一种利用大语言模型创建自动化的对话代理,” *个人与普适计算*, vol. 28, pp. 931–946, 2024.
- [12] T. Calò and L. De Russis, “通过多模态指令歧义增强智能家居交互,” vol. 28, 1000, 2024 6, 985no. p. –.
- [13] C. G. Correa, M. K. Ho, F. Callaway, N. D. Daw, and T. L. Griffiths, “人类通过权衡效用与计算成本来分解任务,” *PLoS计算生物学*, vol. 19, no. 6, p. e1011087, 2023.
- [14] M. L’ovd’én, B. Garzon, and U. Lindenberger, “人类技能学习：扩展、探索、选择与优化,” *行为科学新见*, vol. 36, pp. 163–168, 2020.
- [15] X. Chen, S. Chen, Z. Jin, H. Bian, Z. Chen, and H. Li, “在智能家居中表达需求：终端用户最喜欢的方式是什么,” *ACM人机交互汇刊*, vol. 32, 38, 2025 16, 1no. pp. –.
- [16] T.-H. K. Huang, A. Azaria, and J. P. Bigham, “Instructablecrowd: 通过与人群对话创建If-Then规则,” in *2016年CHI人机交互系统会议论文集 (CHI EA’16)*, 2016.
- [17] G. Campagna, R. Ramesh, S. Xu, M. Fischer, and M. S. Lam, “Almond: 一个开放的、众包的、保护隐私的、可编程虚拟助手的架构,” in *国际万维网会议委员会 (IW3C2)*, 2017.

- [18] J. Jaihar, N. Lingayat, P. S. Vijaybhai, G. Venkatesh, and K. P. Upla, “使用机器学习算法的智能家居自动化,” in 新兴技术国际会议 (INCENT), 2020. [19] S. Kim and I.-Y. Ko, “一种在物联网环境中修改服务混搭的对话式方法,” in 计算机系统中的人为因素会议 (CHI’22), 2022. [20] G. Simone and P. Fabio, “一种用于创建灵活日常自动化的对话代理,” in 2022年高级视觉界面国际会议论文集 (AVI 2022), 2022. [21] F. Corno, L. D. Russis, and A. M. Roffarello, “从用户意图到物联网中的If-Then规则,” 信息系统汇刊 (TOIS), vol. 39, no. 53, pp. 1–33, 2021. [22] S. Lee, J. Choi, J. Lee, M. H. Wasi, H. Choi, S. Y. Ko, S. Oh, and I. Shin, “MobileGPT: 通过类人应用记忆增强大语言模型以实现移动任务自动化,” in 第30届移动计算与网络国际年会论文集, 2024. [23] H. Wen, Y. Li, G. Liu, S. Zhao, T. Yu, T. J.-J. Li, S. Jiang, Y. Liu, Y. Zhang, and Y. Liu, “AutoDroid: Android中基于大语言模型的任务自动化,” in 第24届移动计算与网络国际年会论文集, 2024. [24] C. Zhang, Z. Yang, J. Liu, Y. Li, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu, “AppAgent: 作为智能手机用户的多模态智能体,” in 2025年CHI人机交互系统会议论文集, 2025. [25] J. Lee, D. Lee, C. Choi, Y. Im, J. Wi, K. Heo, S. Oh, S. Lee, and I. Shin, “通过基于逻辑的操作验证保障移动GUI智能体安全,” arXiv预印本 arXiv:2503.18492, 2025. [26] H. Wen, S. Tian, B. Pavlov, W. Du, Y. Li, G. Chang, S. Zhao, J. Liu, Y. Liu, Y.-Q. Zhang et al., “AutoDroid-v2: 通过代码生成提升基于SLM的GUI智能体,” arXiv预印本 arXiv:2412.18116, 2024. [27] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders et al., “WebGPT: 基于浏览器辅助、结合人类反馈的问答,” arXiv预印本 arXiv:2112.09332, 2021. [28] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: 一个用于构建自主智能体的真实网络环境,” 2024, arXiv:2307.13854. [29] W. Hong, W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Dong, M. Ding et al., “CogAgent: 用于GUI智能体的视觉语言模型,” in IEEE/CVF计算机视觉与模式识别会议论文集, 2024, pp. 14 281–14 290. [30] A. Tomar, H. Liang, I. Bhattacharya, N. Larios, and F. Carbone, “Cybernaut: 迈向可靠的网络自动化,” arXiv预印本 arXiv:2508.16688, 2025. [31] D. Gao, L. Ji, Z. Bai, M. Ouyang, P. Li, D. Mao, Q. Wu, W. Zhang, P. Wang, X. Guo, H. Wang, L. Zhou, and M. Z. Shou, “AssistGUI: 面向任务的桌面图形用户界面自动化,” 2024, arXiv:2312.13108. [32] R. Niu, J. Li, S. Wang, Y. Fu, X. Hu, X. Leng, H. Kong, Y. Chang, and Q. Wang, “ScreenAgent: 一种视觉语言模型驱动的计算机控制智能体,” arXiv预印本 arXiv:2402.07945, 2024. [33] C. Zhang, H. Huang, C. Ni, J. Mu, S. Qin, S. He, L. Wang, F. Yang, P. Zhao, C. Du et al., “UFO2: 桌面智能体操作系统,” arXiv预印本 arXiv:2504.14603, 2025. [34] D. Zhao, L. Ma, S. Wang, M. Wang, and Z. Lv, “COLA: 一个用于Windows UI任务自动化的可扩展多智能体框架,” arXiv预印本 arXiv:2503.09263, 2025. [35] 三星, “SmartThings开发者,” <https://developer.smarthings.com/docs/api/public>, 2025, 访问日期: 2025-09-03. [36] F. Corno, L. De Russis, and A. M. Roffarello, “一种面向物联网终端用户开发的高层方法,” in 2017年CHI人机交互系统会议扩展摘要集, 2017. [37] OpenAI, “你好, GPT-4o,” <https://openai.com/ko-KR/index/hello-gpt-4o/>, 2025, 访问日期: 2025-07-11. [38] —, “向量嵌入,” <https://platform.openai.com/docs/guides/embeddings>, 2025, 访问日期: 2025-08-29. [39] 三星, “SmartThings Edge Driver,” <https://developer.smarthings.com/docs/devices/hub-connected/lan>, 2025, 访问日期: 2025-08-28. [40] D. Rivkin, “SAGE,” <https://github.com/SAIC-MONTREAL/SAGE>, 2025, 访问日期: 2025-09-05. [41] OpenAI, “介绍GPT-5.2,” <https://openai.com/index/introducing-gpt-5-2/>, 2025, 访问日期: 2025-12-29. [42] 谷歌, “Gemini 3 Flash,” <https://ai.google.dev/gemini-api/docs/models?hl=ko#gemini-3-flash>, 2025, 访问日期: 2025-12-29. [43] Anthropic, “介绍Claude Opus 4.5,” <https://www.anthropic.com/news/claude-opus-4-5>, 2025, 访问日期: 2025-12-29.